

wpipe at the center of your pipelines

Real parallelism, without spinning up a cluster

The context

Your pipeline has 3 steps that don't depend on each other. Today, in a sequential script, they run one after another wasting time. In production that means: more compute hours, more RAM used, more waiting.

```
from wpipe import Pipeline, step, Parallel
```

Why engineers shift to wpipe

- Lower end-to-end latency in your flows.
- Less infrastructure: no external worker pool needed.
- Parallel orchestration that stays deterministic and traceable.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2, checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="pipeline.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]}

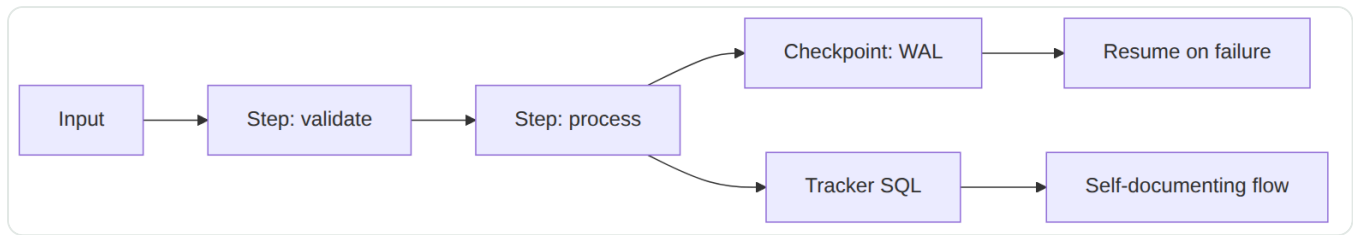
# Python-native orchestration: checkpoints, retries, self-documenting.
```

At a glance

Scenario	Recommended
I/O tasks (network, DB, files)	Threads (`use_processes=False`)
Heavy CPU tasks (pandas, ML)	Processes, with GIL bypass

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **wpipe** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.